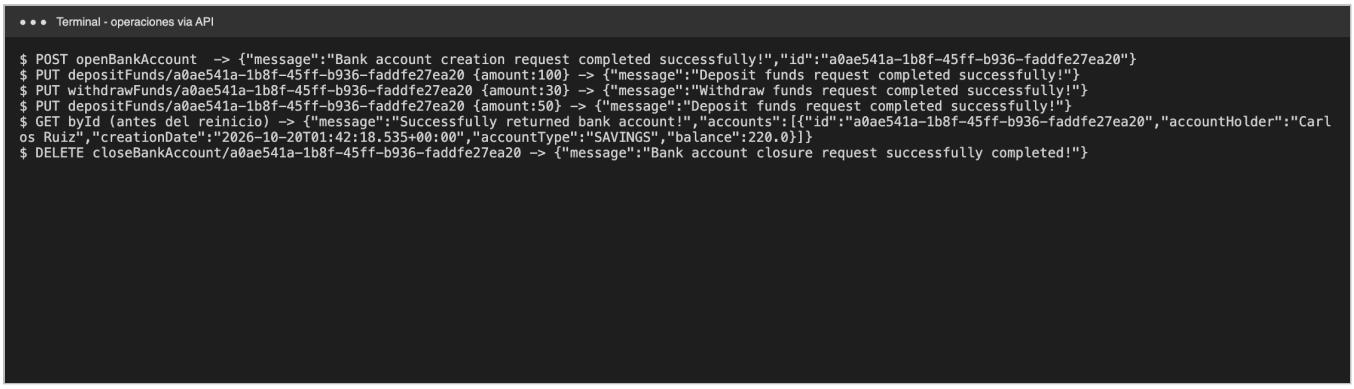
Prueba de reproducción de eventos

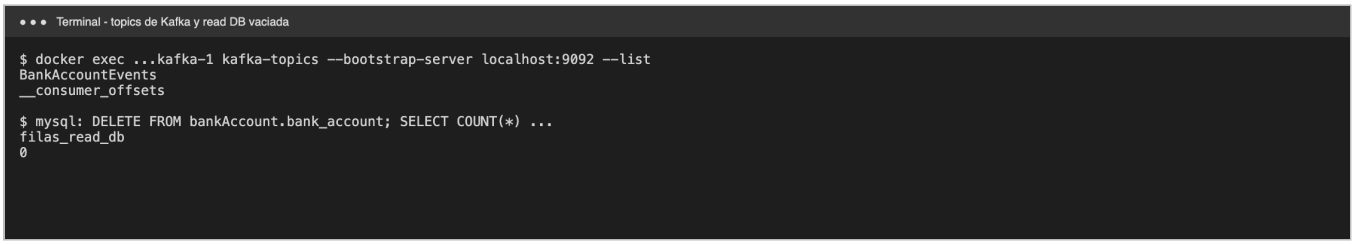
1. Operaciones sobre una cuenta nueva (Carlos Ruiz)

Abrir (100) → depositar 100 → retirar 30 → depositar 50 → consulta (balance 220) → cerrar. Genera 5 eventos (v0 a v4).



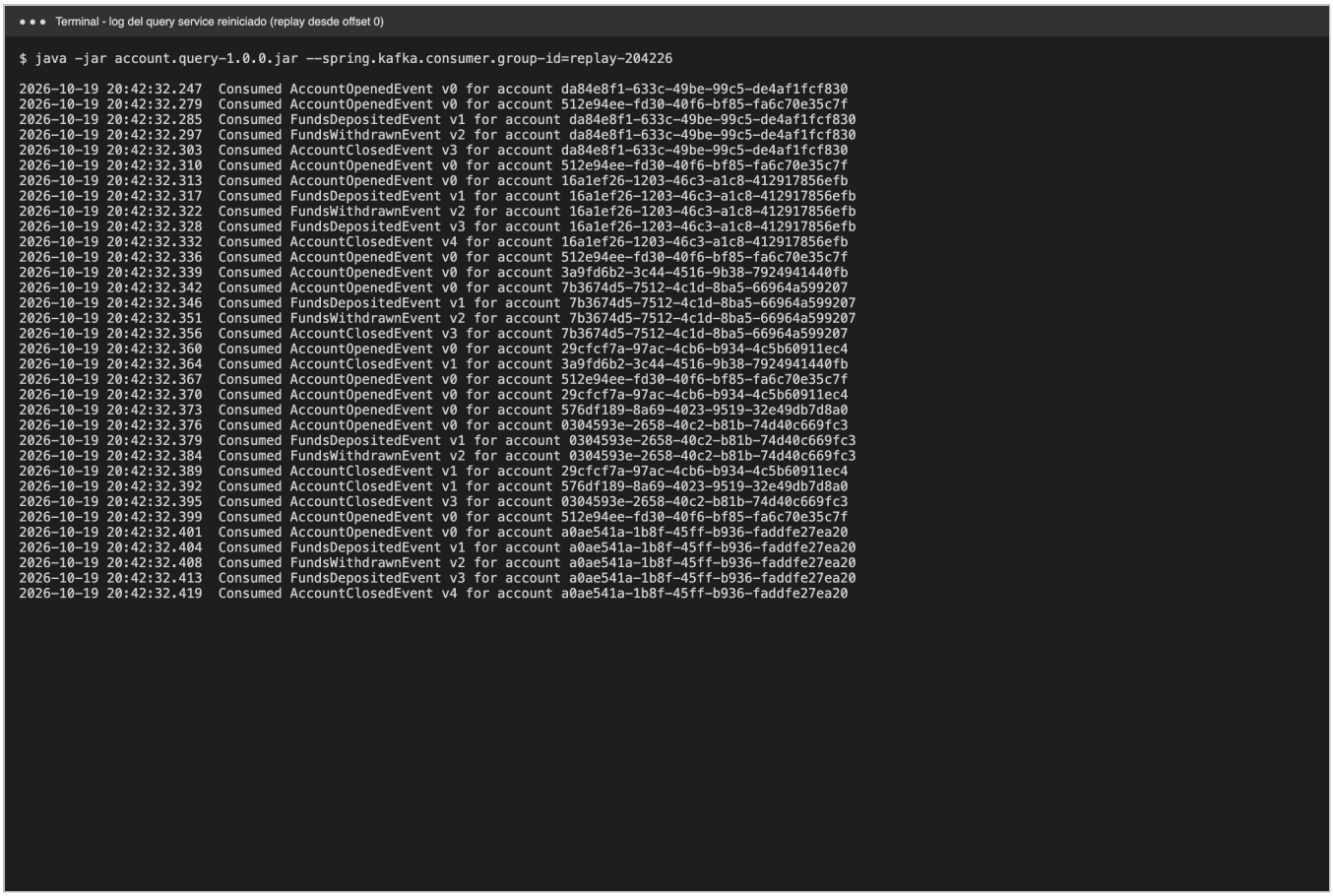
2. Topic único y read DB vaciada

Solo existe `BankAccountEvents` (además de `__consumer_offsets`). Antes del reinicio se detuvo el query service y se vació la tabla del read DB (0 filas).



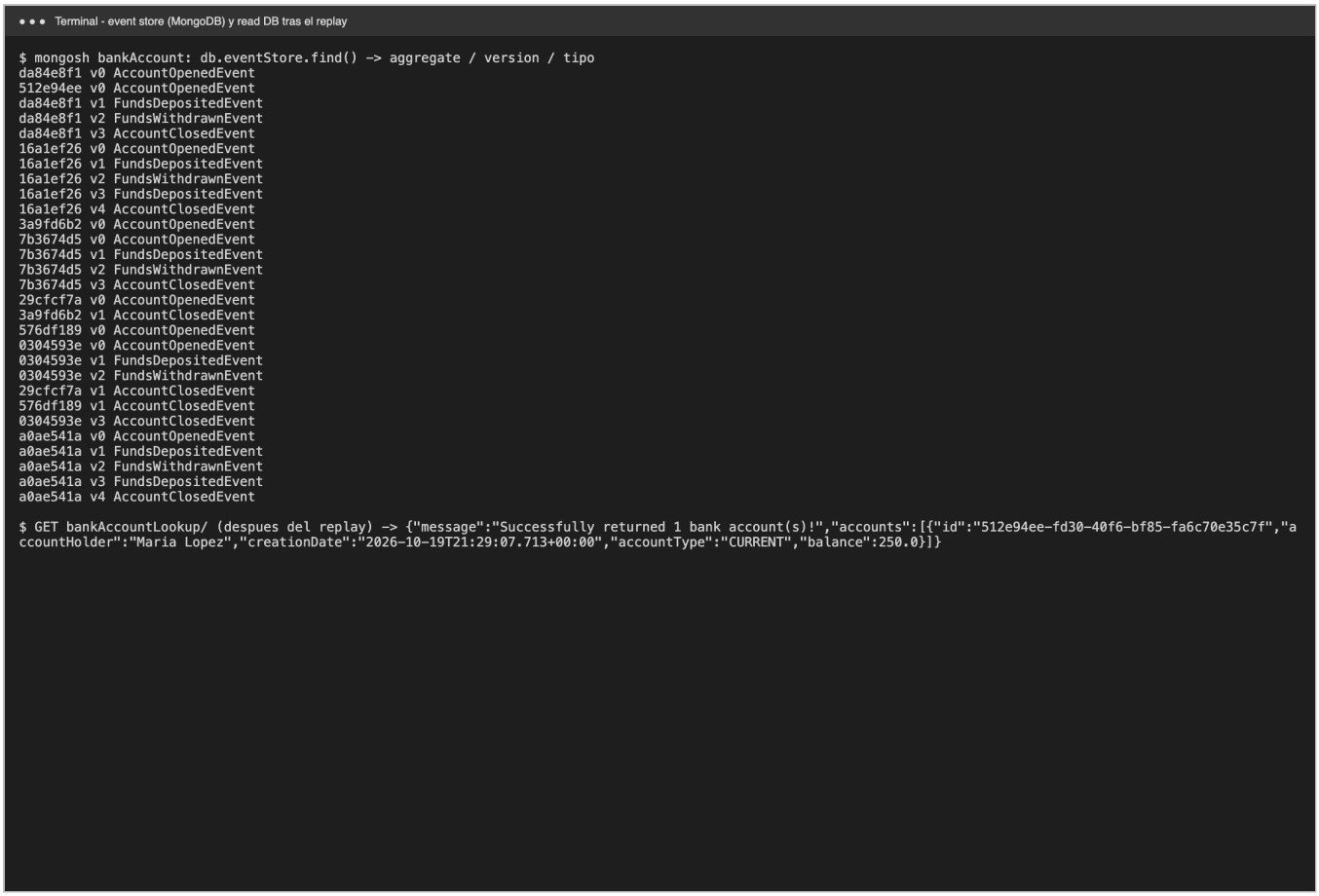
3. Query service reiniciado: eventos reproducidos

Se reinició con un consumer group nuevo (`auto-offset-reset: earliest`), por lo que lee el topic completo desde el offset 0. Cada línea es un evento consumido. Para cada cuenta las versiones salen siempre en orden creciente (v0, v1, v2…): Opened → Deposited → Withdrawn → … → Closed. Las líneas repetidas de `AccountOpenedEvent` de la cuenta 512e94ee (Maria Lopez) son republicaciones de RestoreReadDb, ejecutado varias veces durante las pruebas del endpoint.



4. Coincidencia con el event store y estado final

El orden de consumo coincide con el orden de escritura en MongoDB (aggregate, versión, tipo). Verificación automática sobre el log: 9 cuentas, ninguna con versiones fuera de orden. El read DB reconstruido contiene solo la cuenta activa (Maria Lopez, 250.0); las cerradas no aparecen.



5. Por qué el orden importa en CQRS y Event Sourcing

En Event Sourcing el estado de una cuenta es el resultado de aplicar sus eventos en secuencia, y en CQRS el read model es una proyección de ese flujo. Aplicar eventos fuera de orden (un depósito antes de la apertura, un evento después del cierre) hace que la proyección diverja del event store y que reconstruirla no sea determinista. Con un único topic y key por cuenta, Kafka mantiene el orden por agregado, que es el que garantiza la consistencia de cada cuenta. La reproducción desde el offset 0 demuestra que el read model puede reconstruirse completo y correctamente.